

Tutoriel : Réalisation de tests unitaires avec pytest en Python

Nom : [Votre Nom]

Classe : [Votre Classe]

École : [Votre École]

Module : Programmation Python

Date : 28 juin 2025

Table des matières

1	Introduction	2
1.1	Cible	2
1.2	Objectif visé	2
1.3	Prérequis	2
2	Plan du cours	2
3	Introduction aux tests unitaires	2
3.1	Pourquoi utiliser des tests unitaires?	3
4	Présentation de pytest	3
5	Structuration d'un projet Python pour les tests unitaires	3
5.1	Explications	3
6	Étapes de création des tests avec pytest	3
7	Exemple pratique : Projet de calculatrice	4
7.1	Code source (<code>src/calculatrice.py</code>)	4
7.2	Tests unitaires (<code>tests/test_calculatrice.py</code>)	4
7.3	Exécution des tests	4
8	Bonnes pratiques et astuces	5
9	Bibliographie et webographie	5

1 Introduction

Ce document est un support de cours destiné à enseigner la réalisation de tests unitaires en Python à l'aide du framework `pytest`. Il s'agit d'un guide pratique pour les développeurs débutants souhaitant apprendre à écrire et exécuter des tests unitaires pour garantir la fiabilité de leur code.

1.1 Cible

Ce cours s'adresse aux étudiants en informatique, aux développeurs Python débutants ou intermédiaires, et à toute personne souhaitant approfondir ses compétences en tests unitaires dans un contexte de développement logiciel.

1.2 Objectif visé

L'objectif est de permettre aux apprenants de :

- Comprendre l'importance des tests unitaires.
- Maîtriser l'utilisation de `pytest` pour écrire et exécuter des tests.
- Structurer un projet Python avec des tests unitaires.
- Appliquer les bonnes pratiques pour garantir un code robuste et maintenable.

1.3 Prérequis

Pour suivre ce tutoriel, il est requis de :

- Connaître les bases de la programmation Python (fonctions, classes, modules).
- Avoir Python (version 3.8 ou supérieure) installé sur votre machine.
- Installer `pytest` via la commande `pip install pytest`.
- Disposer d'un éditeur de code (par exemple, VS Code, PyCharm) et d'un terminal.

2 Plan du cours

1. Introduction aux tests unitaires
2. Présentation de `pytest`
3. Structuration d'un projet Python pour les tests unitaires
4. Étapes de création des tests avec `pytest`
5. Exemple pratique : Projet de calculatrice
6. Bonnes pratiques et astuces
7. Bibliographie et webographie

3 Introduction aux tests unitaires

Les tests unitaires consistent à tester individuellement les unités de code (fonctions, méthodes, classes) pour s'assurer qu'elles fonctionnent comme prévu. Ils permettent de détecter rapidement les erreurs, de faciliter la maintenance et d'améliorer la qualité du code.

3.1 Pourquoi utiliser des tests unitaires ?

- **Fiabilité** : Vérifier que chaque composant fonctionne correctement.
- **Maintenance** : Faciliter la détection des régressions lors de modifications.
- **Documentation** : Les tests servent de documentation vivante du code.

4 Présentation de pytest

Pytest est un framework de test puissant et flexible pour Python. Il est apprécié pour sa simplicité, sa lisibilité et ses nombreuses fonctionnalités, telles que :

- Syntaxe intuitive pour écrire des tests.
- Détection automatique des fichiers et fonctions de test.
- Support des assertions Python standard sans modules supplémentaires.
- Plugins pour étendre ses fonctionnalités.

Liens utiles :

- Documentation officielle : <https://docs.pytest.org/>
- Installation : <https://docs.pytest.org/en/stable/getting-started.html>

5 Structuration d'un projet Python pour les tests unitaires

Un projet Python bien structuré facilite l'écriture et l'exécution des tests. Voici une structure typique :

```
1 mon_projet/  
2   src/  
3       calculatrice.py    % Code source  
4   tests/  
5       test_calculatrice.py % Tests unitaires  
6   requirements.txt      % Dépendances  
7   README.md            % Documentation
```

5.1 Explications

- **src/** : Contient le code source de l'application.
- **tests/** : Contient les fichiers de test, préfixés par `test.requirements.txt` : *Liste les dépendances, par exemple*

6 Étapes de création des tests avec pytest

Voici les étapes pour créer et exécuter des tests avec pytest :

1. **Installer pytest** : Exécutez `pip install pytest` dans votre environnement.
2. **Créer un module à tester** : Écrivez le code de votre application dans un dossier `src/`.
3. **Écrire les tests** : Créez des fichiers de test dans `tests/` avec le préfixe `test`. **Exécuter les tests** : *Depuis la racine du projet, exécutez `pytest` ou `pytest -v` pour une affiche détaillée.*
4. **Analyser les résultats** : Pytest affiche les tests réussis (`.`), échoués (`F`), ou ignorés (`s`).

7 Exemple pratique : Projet de calculatrice

Voici un exemple concret d'une calculatrice avec des tests unitaires.

7.1 Code source (src/calculatrice.py)

```
1 def addition(a, b):
2     return a + b
3
4 def soustraction(a, b):
5     return a - b
6
7 def multiplication(a, b):
8     return a * b
9
10 def division(a, b):
11     if b == 0:
12         raise ValueError("Division par zéro interdite")
13     return a / b
```

7.2 Tests unitaires (tests/test_calculatrice.py)

```
1 from src.calculatrice import addition, soustraction, multiplication,
   division
2 import pytest
3
4 def test_addition():
5     assert addition(2, 3) == 5
6     assert addition(-1, 1) == 0
7     assert addition(0, 0) == 0
8
9 def test_soustraction():
10    assert soustraction(5, 3) == 2
11    assert soustraction(3, 5) == -2
12    assert soustraction(0, 0) == 0
13
14 def test_multiplication():
15    assert multiplication(2, 3) == 6
16    assert multiplication(-2, 3) == -6
17    assert multiplication(0, 5) == 0
18
19 def test_division():
20    assert division(6, 2) == 3
21    assert division(5, 2) == 2.5
22    with pytest.raises(ValueError):
23        division(10, 0)
```

7.3 Exécution des tests

Pour exécuter les tests, placez-vous à la racine du projet et exécutez :

```
1 pytest -v
```

Résultat attendu :

```
1 tests/test_calculatrice.py::test_addition PASSED
2 tests/test_calculatrice.py::test_soustraction PASSED
3 tests/test_calculatrice.py::test_multiplication PASSED
4 tests/test_calculatrice.py::test_division PASSED
```

8 Bonnes pratiques et astuces

- Nommez les fichiers de test avec le préfixe `test` et les fonctions de test d'une manière descriptive. Testez les fonctions.
- Utilisez `pytest.mark` pour organiser les tests (par exemple, `@pytest.mark.slow`).
- Exécutez `pytest -cov` pour mesurer la couverture de code (nécessite `pip install pytest-cov`).
- Gardez les tests indépendants et rapides.

9 Bibliographie et webographie

- **Bibliographie :**
 - Hunter, B. (2021). *Python Testing with pytest*. Pragmatic Bookshelf.
- **Webographie :**
 - Documentation officielle de pytest :
 - Tutoriel pytest :